

pathos

Fewer wrong turns, more right answers.

CSL 2120 – Data Structures & Algorithms

Shreshth Dhimole	B25EE1037
Madhav Saxena	B25EE1019
Moksh Sharma	B25EC1025

Problem Statement

Pathfinding algorithms are often demonstrated on grids, but real roads have unequal lengths, irregular junctions and one-way restrictions. These examples do not clearly show how algorithms behave on an actual city map. We want to compare routes on real road networks and explore how to plan a short tour through several selected stops.

Proposed Solution

Abstract. We propose **pathos**, an interactive urban routing and Traveling Salesperson Problem (TSP) visualizer built around OpenStreetMap data. Users will select locations on a map, choose a routing algorithm and view both the route and an animation of the search. The aim is to make graph algorithms easier to understand through real roads rather than artificial grids.

Python and OSMnx will prepare the road network, while a C++ engine will store it as an adjacency list and run BFS, Dijkstra and A*. BFS will find a route with the fewest road segments; Dijkstra and A* will search for the shortest distance. A* will use a geographic distance estimate to guide its search. The comparison will show running time, explored nodes and route length.

For multiple stops, the TSP module will seek a short tour that visits every selected stop and returns to the start. It will use shortest-path road distances between stops, build an initial tour with nearest-neighbor and improve the stop order using 2-opt. These methods provide a practical approximation rather than a guaranteed optimal tour. One-way roads and unreachable stops will be handled when checking routes.

A FastAPI server will connect the routing engine to a React interface built with Vite and Leaflet.js. The engine will return route details and recorded exploration steps, which the map will replay as an animation. This keeps the algorithm work separate from the interface while giving users a clear way to compare searches and multi-stop tours.

Technology Stack

Backend and Algorithms

C++ — Store the road graph and run routing and TSP algorithms.
Python + FastAPI — Connect the routing engine to the web interface.
CMake — Build the C++ engine.

Map Data

OSMnx + Pandas — Extract and prepare OpenStreetMap road data.

Frontend and Visualization

React + Vite — Build the web interface and its controls.
Leaflet.js — Display the map and let users select locations.
D3.js / Mapbox GL JS — Animate search steps and highlight routes.
npm — Manage frontend packages and build scripts.

Team Responsibilities

Member	Responsibility
A	Frontend map, location selection, search animation and interface testing.
B	Road data preparation, FastAPI server, engine integration and error handling.
C	Routing algorithms, TSP heuristics, performance measurement and algorithm testing.

All members will contribute to integration testing and documentation, with implementation effort balanced across the three roles.